Human Verification: `TrustBoundary`

Trust-Labelled Trees, the Boundary Rule, and Duplicate-Stability

Machine-generated from `lean/TrustBoundary.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `TrustBoundary` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean.

The module links proof registries (finitely-branching rose trees with trust labels) to the trust-system theory of `Trust.lean`. The main structural result is *duplicate-stability*: if the global boundary rule holds for a tree t, it also holds for every subtree $t.\mathrm{sub}(p)$. The deployed trust chain (`TrustLevel`, 7 levels) and the loop experiment's chain (`LoopTrustLevel`, 6 levels) are also instantiated here.

Contents

1 Overview **2**

2 Trust labels and labelled trees **2**
2.1 Structure: `TrustLabel` . . . 2
2.2 Inductive type: `LTree` . . . 2
2.3 Definition: `LTree.lab` . . . 3

3 Positions, sub-trees, and paths **3**
3.1 Inductive type: `LTree.LPos` . . . 3
3.2 Definition: `LTree.sub` . . . 3
3.3 Definition: `LTree.shift` . . . 4

4 Directed-container axioms for `LTree` **4**
4.1 Definitions: `shift_root`, `sub_shift`, `shift_assoc` . . . 4

5 The trust registry as a directed container **5**
5.1 Definition: `trustRegistry` . . . 5

6 The boundary rule **5**
6.1 Definition: `LTree.localOk` . . . 5
6.2 Definition: `LTree.Ok` . . . 6

7 Duplicate-stability **6**
7.1 Theorem: `LTree.ok_sub` . . . 6
7.2 Theorem: `LTree.ok_iff_all_sub` . . . 7

8 The deployed trust chain **7**
8.1 Inductive type: `Trust.TrustLevel` . . . 7
8.2 Definition: `TrustLevel.rank` and `instance : Chain TrustLevel` . . . 7

8.3 Abbreviation: `Trust.RegistryOk` and theorem `registryOk_sub` 8

9 The loop experiment's trust chain **8**
9.1 Inductive type: `LoopTrust.LoopTrustLevel` 8
9.2 Definition: `LoopTrustLevel.rank` and `instance : Chain LoopTrustLevel` . . 9
9.3 Abbreviation: `LoopRegistryOk` and theorem `loopRegistryOk_sub` 9

10 Translation conventions **9**

1 Overview

Deployed proof registries are finitely-branching rose trees (`LTree`) whose nodes carry a trust label. The boundary rule (`registry_validate.py`) is node-local: a node may claim `proved` or above only if every non-dead child claims `proved` or above. Equivalently, the rule is a *coKleisli predicate*: apply the local check at every node produced by the comonad operation `duplicate` (= `sub`).

The module proves:

1. **Duplicate-stability** (`ok_sub`): the global boundary rule propagates to every subtree.
2. **The deployed chain** (`TrustLevel`): a `Chain` instance for the seven-level registry chain, so the certificate lemma and lax transport from `Trust.lean` apply to real registry labels.
3. **The loop experiment's chain** (`LoopTrustLevel`): a six-level variant for the Warnaar positivity loop.

2 Trust labels and labelled trees

2.1 Structure: `TrustLabel`

Definition 2.1 (Trust label)**.** A **trust label** is a pair: a trust level $\ell \in T$ (a level in the chain T) and a boolean `dead` flag. Dead-end nodes are *exempt* from the boundary rule; `dead = true` means the approach was abandoned. An order relation cannot express an exemption, which is why `dead` is a separate flag rather than a chain element.

Lean.
```
structure TrustLabel (T : Type) where
  trust : T
  dead  : Bool
```

Haskell.
```
data TrustLabel t = TrustLabel
  { trust :: t
  , dead  :: Bool
  }
```

2.2 Inductive type: `LTree`

Definition 2.2 (Labelled rose tree)**.** A **trust-labelled finitely-branching rose tree** has a `TrustLabel T` at every node and an explicit arity $n : \mathbb{N}$ with children indexed by $\mathrm{Fin}(n)$. This is the shape of a deployed registry (`proofs/registry/*.json`).

Lean.

```
inductive LTree (T : Type) where
  | node : TrustLabel T → (n : Nat) → (Fin n → LTree T) → LTree T
```

Haskell.

```
data LTree t = Node (TrustLabel t) [LTree t]
-- (Using a list of children rather than a Fin-indexed function)
```

Fidelity note. In Lean the arity n is an explicit `Nat` and children are indexed by `Fin n`, giving a *dependent* function type. In Haskell we use a list, losing the arity-indexed dependency.

2.3 Definition: `LTree.lab`

Definition 2.3 (Root label)**.** lab(t) extracts the trust label at the root of t. Under the comonad, lab(t.sub(p)) reads the label at position p.

Lean.

```
def LTree.lab : LTree T → TrustLabel T
  | .node l _ _ ⇒ l
```

Haskell.

```
lab :: LTree t → TrustLabel t
lab (Node l _) = l
```

3 Positions, sub-trees, and paths

3.1 Inductive type: `LTree.LPos`

Definition 3.1 (Position / path)**.** A **position** in t : `LTree` T is a path from the root to a node. It is either `here` (the root) or a child index i followed by a position in the i-th child.

Lean.

```
inductive LPos : LTree T → Type where
  | here  : LPos t
  | child : (i : Fin n) → LPos (children i)
              → LPos (.node l n children)
```

Haskell.

```
data LPos t
  = Here
  | Child Int (LPos t)    -- approximate: index is Int, not Fin n
```

3.2 Definition: `LTree.sub`

Definition 3.2 (Sub-tree)**.** t.sub(p) is the sub-tree rooted at position p.

Lean.

```
def sub : (t : LTree T) → LPos t → LTree T
  | t, .here ⇒ t
  | .node _ _ children, .child i p ⇒ (children i).sub p
```

Haskell.

```
sub :: LTree t → LPos (LTree t) → LTree t
sub t Here            = t
sub (Node _ cs) (Child i p) = sub (cs !! i) p
```

3.3 Definition: `LTree.shift`

Definition 3.3 (Path concatenation)**.** $t.\mathrm{shift}(p, q)$ concatenates path p with path q (where q is a position in $t.\mathrm{sub}(p)$), giving a position in t.

Lean.
```
def shift : (t : LTree T) → (p : LPos t)
              → LPos (t.sub p) → LPos t
  | _, .here, q ⇒ q
  | .node _ _ children, .child i p, q ⇒
      .child i ((children i).shift p q)
```

Haskell.
```
shift :: LTree t → LPos (LTree t) → LPos (LTree t) → LPos (LTree t)
shift _ Here q = q
shift (Node _ cs) (Child i p) q = Child i (shift (cs !! i) p q)
```

4 Directed-container axioms for `LTree`

4.1 Definitions: `shift_root`, `sub_shift`, `shift_assoc`

The following three lemmas confirm that `LTree` with `sub` and `shift` satisfies the directed-container axioms D3, D4, D5.

Theorem 4.1 (D3: shift to root)**.** For every tree t and position p,

$$t.\mathrm{shift}(p, \mathrm{here}) = p.$$

Lean.
```
def shift_root : (t : LTree T) → (p : LPos t) →
    t.shift p .here = p
```

Theorem 4.2 (D4: sub-trees nest along shift)**.** For every t, $p : \mathrm{LPos}(t)$, $q : \mathrm{LPos}(t.\mathrm{sub}(p))$,

$$t.\mathrm{sub}(t.\mathrm{shift}(p, q)) = (t.\mathrm{sub}(p)).\mathrm{sub}(q).$$

Lean.
```
def sub_shift : (t : LTree T) → (p : LPos t)
    → (q : LPos (t.sub p)) →
    t.sub (t.shift p q) = (t.sub p).sub q
```

Theorem 4.3 (D5: shift is associative)**.** For t, p, q, r at compatible types (with transport along D4),

$$t.\mathrm{shift}(t.\mathrm{shift}(p, q),\, r) = t.\mathrm{shift}(p,\, (t.\mathrm{sub}(p)).\mathrm{shift}(q, r)).$$

Lean.
```
def shift_assoc : (t : LTree T) → (p : LPos t)
    → (q : LPos (t.sub p))
    → (r : LPos ((t.sub p).sub q)) →
    t.shift (t.shift p q) ((t.sub_shift p q).symm ⇒ r) =
      t.shift p ((t.sub p).shift q r)
```

Fidelity note. The `=>` in D5 is Lean transport notation: $(t.\mathrm{sub_shift}\, p\, q)^{-1} \blacktriangleright r$ re-types $r : \mathrm{LPos}((t.\mathrm{sub}(p)).\mathrm{sub}(q))$ as $r : \mathrm{LPos}(t.\mathrm{sub}(t.\mathrm{shift}(p, q)))$ via the equality from D4. In Haskell this coercion is invisible (all trees have the same type), but in Lean's dependent type theory the transport is an explicit proof term. The listings use `=>` as an ASCII approximation of the transport operator.

Haskell.

```haskell
-- shift_root:  shift t p Here = p
-- sub_shift:   sub t (shift t p q) = sub (sub t p) q
-- shift_assoc: shift t (shift t p q) r = shift t p (shift (sub t p) q
    r)
-- All hold by structural induction on p.
```

5 The trust registry as a directed container

5.1 Definition: `trustRegistry`

Definition 5.1 (Trust registry directed container)**.** The trust-labelled rose trees form a directed container with

$$\begin{aligned} \text{Shape} &= \texttt{LTree}\,T, \\ \text{Pos} &= \texttt{LTree.LPos}, \\ \text{root} &= \lambda t.\,\texttt{here}, \\ \text{sub} &= \texttt{LTree.sub}, \\ \text{shift} &= \texttt{LTree.shift}. \end{aligned}$$

Axioms D1–D5 are provided by `rfl`, `sub_shift`, `shift_root`, and `shift_assoc` respectively (D1 and D2 hold definitionally).

Lean.

```lean
def trustRegistry (T : Type) : DirectedContainer where
  Shape := LTree T
  Pos   := LTree.LPos
  root  := fun _ ⇒ .here
  sub   := LTree.sub
  shift := LTree.shift
  d1    := fun _ ⇒ rfl
  d4    := LTree.sub_shift
  d2    := fun _ _ ⇒ rfl
  d3    := LTree.shift_root
  d5    := LTree.shift_assoc
```

Haskell.

```haskell
-- trustRegistry :: DirectedContainer (LTree T) (LPos (LTree T))
-- root t = Here
-- sub   = LTree.sub
-- shift = LTree.shift
-- D1--D5 hold by structural induction.
```

6 The boundary rule

6.1 Definition: `LTree.localOk`

Definition 6.1 (Node-local boundary rule)**.** At threshold $\theta : T$, a tree $t = \texttt{node}\,l\,n\,\text{children}$ satisfies the **local boundary rule** if: whenever $\theta \le l.\text{trust}$ (this node claims above threshold), every non-dead child i satisfies $\theta \le \text{children}(i).\text{lab.trust}$.

Formally:

$$\text{localOk}(\theta, t) \iff \theta \le t.\text{lab.trust} \Rightarrow \forall\, i : \text{Fin}(n),\ \text{children}(i).\text{lab.dead} = \text{true} \vee \theta \le \text{children}(i).\text{lab.trust}.$$

This is precisely what `registry_validate.py` checks at each node.

Lean.

```
def localOk (theta : T) : LTree T → Prop
  | .node l n children ⇒
      theta ≤ l.trust → ∀ i : Fin n,
        (children i).lab.dead = true \/ theta ≤ (children i).lab.
           trust
```

Haskell.

```
localOk :: (Ord t) ⇒ t → LTree t → Bool
localOk theta (Node l children)
  | trust l >= theta = all (\c → dead (lab c) || trust (lab c) >=
     theta) children
  | otherwise        = True
```

6.2 Definition: `LTree.Ok`

Definition 6.2 (Global boundary rule)**.** The **global boundary rule** holds for t at threshold θ if the local check holds at every position (i.e., at every sub-tree produced by `duplicate`):

$$\mathrm{Ok}(\theta, t) \iff \forall\, p : \mathrm{LPos}(t),\ \mathrm{localOk}(\theta,\, t.\mathrm{sub}(p)).$$

Lean.

```
def Ok (theta : T) (t : LTree T) : Prop :=
  ∀ p : LPos t, (t.sub p).localOk theta
```

Haskell.

```
ok :: (Ord t) ⇒ t → LTree t → Bool
ok theta t = all (\p → localOk theta (sub t p)) (allPositions t)
  where allPositions = ...   -- enumerate all positions recursively
```

7 Duplicate-stability

7.1 Theorem: `LTree.ok_sub`

Theorem 7.1 (Duplicate-stability of the trust boundary)**.** If the global boundary rule holds for t at threshold θ, then it also holds for every subtree:

$$\mathrm{Ok}(\theta, t) \;\Rightarrow\; \forall\, p : \mathrm{LPos}(t),\ \mathrm{Ok}(\theta,\, t.\mathrm{sub}(p)).$$

The proof is a one-line transport along D4: a position q in $t.\mathrm{sub}(p)$ corresponds to the position $t.\mathrm{shift}(p, q)$ in t; D4 identifies the subtrees; so the check at $(t, t.\mathrm{shift}(p, q))$ *is* the check at $(t.\mathrm{sub}(p), q)$.

Lean.

```
theorem ok_sub (theta : T) (t : LTree T) (h : Ok theta t)
    (p : LPos t) : Ok theta (t.sub p) :=
  fun q ⇒ t.sub_shift p q ⇒ h (t.shift p q)
```

Haskell.

```
-- ok_sub: ok theta t → ok theta (sub t p)
--
-- For any position q in (sub t p):
--   localOk theta (sub (sub t p) q)
--   = localOk theta (sub t (shift t p q))     -- by D4
--   = h (shift t p q)                         -- from Ok theta t
```

Fidelity note. In Lean, the transport `t.sub_shift p q ▶ h (t.shift p q)` coerces the hypothesis along the D4 equality. In Haskell (or informal mathematics) this coercion is invisible because the identity of the tree doesn't depend on its type index.

7.2 Theorem: `LTree.ok_iff_all_sub`

Theorem 7.2 (Ok iff all sub-trees are Ok)**.**

$$\mathrm{Ok}(\theta, t) \iff \forall\, p : \mathrm{LPos}(t),\ \mathrm{Ok}(\theta,\ t.\mathrm{sub}(p)).$$

The forward direction is `ok_sub`; the backward direction reads the root component (`here`).

Lean.

```
theorem ok_iff_all_sub (theta : T) (t : LTree T) :
    Ok theta t <-> ∀ p : LPos t, Ok theta (t.sub p)
```

Haskell.

```
-- ok theta t <=> all (\p -> ok theta (sub t p)) (allPositions t)
-- Forward: ok_sub; Backward: apply at Here (sub t Here = t).
```

8 The deployed trust chain

8.1 Inductive type: `Trust.TrustLevel`

Definition 8.1 (Deployed trust chain)**.** The seven-level chain used in production registries (schema v3/v4):

`unchecked` < `speculative` < `computed` < `proved` < `peerReviewed` < `published` < `leanVerified`.

`unchecked` is $\bot$, modelling both `in-progress` and `unclassified` (the validator's CAP_RANK -1). `dead-end` is the `dead` flag, not a chain element.

Lean.

```
inductive TrustLevel where
  | unchecked
  | speculative
  | computed
  | proved
  | peerReviewed
  | published
  | leanVerified
deriving DecidableEq
```

Haskell.

```
data TrustLevel
  = Unchecked | Speculative | Computed | Proved
  | PeerReviewed | Published | LeanVerified
  deriving (Eq, Ord, Show)
```

8.2 Definition: `TrustLevel.rank` and `instance : Chain TrustLevel`

Definition 8.2 (Numeric rank and chain instance)**.** The rank assigns `unchecked` $\mapsto 0, \ldots,$ `leanVerified` $\mapsto 6$; the order $a \le b$ is $\mathrm{rank}(a) \le \mathrm{rank}(b)$. The `Chain TrustLevel` instance makes the certificate lemma, attainment, and lax transport from `Trust.lean` available for real registry labels.

Lean.

```
def TrustLevel.rank : TrustLevel → Nat
  | .unchecked    ⇒ 0 | .speculative ⇒ 1
  | .computed     ⇒ 2 | .proved      ⇒ 3
  | .peerReviewed ⇒ 4 | .published   ⇒ 5
  | .leanVerified ⇒ 6
```

```
instance : Chain TrustLevel where
  le a b := a.rank ≤ b.rank
  bot := .unchecked
  -- le_refl, le_trans, le_antisymm, le_total, bot_le, dec all proved
```

Haskell.

```
-- Derived Ord matches the rank ordering.
-- bot = Unchecked.
```

8.3 Abbreviation: `Trust.RegistryOk` and theorem `registryOk_sub`

Definition 8.3 (Deployed boundary rule)**.** The **registry boundary rule** at the deployed threshold `proved`:

$$\mathrm{RegistryOk}(t) \iff \mathrm{Ok}(\texttt{proved}, t).$$

Theorem 8.4 (Deployed duplicate-stability)**.** If $\mathrm{RegistryOk}(t)$ and $p : \mathrm{LPos}(t)$, then $\mathrm{RegistryOk}(t.\mathrm{sub}(p))$.

Lean.

```
abbrev RegistryOk (t : LTree TrustLevel) : Prop :=
  t.Ok TrustLevel.proved

theorem registryOk_sub (t : LTree TrustLevel) (h : RegistryOk t)
    (p : LTree.LPos t) : RegistryOk (t.sub p) :=
  LTree.ok_sub TrustLevel.proved t h p
```

Haskell.

```
registryOk :: LTree TrustLevel → Bool
registryOk t = ok Proved t

-- registryOk_sub: registryOk t ⇒ registryOk (sub t p)
```

9 The loop experiment's trust chain

9.1 Inductive type: `LoopTrust.LoopTrustLevel`

Definition 9.1 (Loop trust chain)**.** The six-level chain for the Warnaar positivity loop experiment:

$$\texttt{unchecked} < \texttt{speculative} < \texttt{computed} < \texttt{proved} < \texttt{verified} < \texttt{leanVerified}.$$

Differs from `TrustLevel` in that `verified` replaces `peerReviewed` and `published`: a `verified` node means an independent verifier (or Robin) hostile-reviewed the proof.

Lean.

```
inductive LoopTrustLevel where
  | unchecked | speculative | computed
  | proved    | verified    | leanVerified
deriving DecidableEq
```

Haskell.

```
data LoopTrustLevel
  = LUnchecked | LSpeculative | LComputed
  | LProved    | LVerified    | LLeanVerified
  deriving (Eq, Ord, Show)
```

9.2 Definition: `LoopTrustLevel.rank` and `instance : Chain LoopTrustLevel`

Definition 9.2 (Numeric rank and chain instance for loop trust)**.** Rank runs from 0 (`unchecked`) to 5 (`leanVerified`). The `Chain LoopTrustLevel` instance brings the full trust-system theory to bear on loop registries.

Lean.

```
def LoopTrustLevel.rank : LoopTrustLevel → Nat
  | .unchecked    ⇒ 0 | .speculative  ⇒ 1
  | .computed     ⇒ 2 | .proved       ⇒ 3
  | .verified     ⇒ 4 | .leanVerified ⇒ 5

instance : Chain LoopTrustLevel where
  le a b := a.rank ≤ b.rank
  bot := .unchecked
  -- le_refl, le_trans, le_antisymm, le_total, bot_le, dec all proved
```

Haskell.

```
-- Derived Ord matches rank ordering; bot = LUnchecked.
```

9.3 Abbreviation: `LoopRegistryOk` and theorem `loopRegistryOk_sub`

Definition 9.3 (Loop boundary rule)**.**

$$\text{LoopRegistryOk}(t) \iff \text{Ok}(\texttt{proved}, t)$$

for t : `LTree LoopTrustLevel`.

Theorem 9.4 (Loop duplicate-stability)**.** LoopRegistryOk(t) implies LoopRegistryOk(t.sub(p)) for every p.

Lean.

```
abbrev LoopRegistryOk (t : Containers.LTree LoopTrustLevel) : Prop :=
  t.Ok LoopTrustLevel.proved

theorem loopRegistryOk_sub (t : Containers.LTree LoopTrustLevel)
    (h : LoopRegistryOk t) (p : Containers.LTree.LPos t) :
    LoopRegistryOk (t.sub p) :=
  Containers.LTree.ok_sub LoopTrustLevel.proved t h p
```

Haskell.

```
loopRegistryOk :: LTree LoopTrustLevel → Bool
loopRegistryOk t = ok LProved t

-- loopRegistryOk_sub: loopRegistryOk t ⇒ loopRegistryOk (sub t p)
```

10 Translation conventions

1. **Dependent type families.** Positions `LPos t` depend on the tree t in Lean; in Haskell all trees have the same position type, losing this dependency.
2. **Transport.** D5 (`shift_assoc`) uses transport along the D4 equality. The listings approximate `|>` (the Lean transport operator) by `=>` (meaning "cast via this proof"); no Haskell counterpart exists.
3. **Duplicate-stability via D4.** The one-line proof of `ok_sub` is $q \mapsto$ `sub_shift p q ▶ h (shift p q)`: a D4 transport applied to the hypothesis. In Haskell, D4 holds computationally and no coercion is needed.

4. **Proof bodies omitted.** Lean checks all proofs.
5. **Namespace structure.** `TrustLabel`, `LTree`, `trustRegistry` are in `Containers`; `TrustLevel` and `RegistryOk` are in `Containers.Trust`; `LoopTrustLevel` and `LoopRegistryOk` are in `Containers.LoopTrust`.